RESEARCH PLAN — INITIAL DRAFT

Informed Bit-Position Pruning for HDC on FPGA

A discriminability-driven Pareto study on Zynq, with cross-subject transferability, for EMG classification — theory, RTL, and publication roadmap

PROJECT	PRIMARY VENUE	BACKUP	DATE
1024HDC	DATE 2027	AICAS · FCCM 2027	May 2026

READ FIRST

Executive summary

The **1024HDC** project already contains a verified 1024-bit binary HDC core (bind + permute) on Zynq with an AXI-Lite slave wrapper and a passing 71 / 71 ModelSim regression. This document is the blueprint that turns that core into a complete **streaming HDC classifier** evaluated on biosignals, with three concrete research contributions absent from prior FPGA-HDC work: a **three-axis (dimension × precision × bit-pruning) Pareto study** on real Zynq energy, an **informed-vs.-random bit pruning comparison** that establishes *why* trained pruning matters, and a **cross-subject mask transferability** study on EMG.

Item	Summary
Goal	Build, measure, and publish a streaming HDC classifier on Zynq with a discriminability-driven Pareto study.
Novelty hook (Hook A)	**Dimension** $D \in \{256, 512, 1024, 2048\}$ · **bundle precision** CNT_W ∈ {3, 4, 5, 6} bits · **bit-position pruning** by training-time discriminability.
Twist 1 — informed vs. random	For every pruning ratio, compare the discriminability-driven mask against random-bit dropout at the same density — the result establishes that bit-position matters, not just bit-count.
Twist 2 — cross-subject transfer	Train the bit-mask on subjects 1–18 of UCI EMG; evaluate it unchanged on subjects 19–36. Either result is publishable (universal vs. per-subject mask).
Application	EMG hand-gesture recognition (UCI 5-class, 36 subjects) · ECG arrhythmia (MIT-BIH, secondary).
New RTL	`item_memory`, `bundle_unit`, `popcount_am`, `pruning_mask`, `encoder_top`, `hdc_stream_wrapper` — all parameterised on D.
Re-used RTL	`xor_permute_top.sv`, `permute_stage.sv` — unchanged, becomes the "verified IP" of the paper.
Software	Bare-metal C on Cortex-A9 + a Python training pipeline that computes the per-bit discriminability score and emits the bit-pruning mask.
Verification	Bit-exact Python golden reference + 10 000 randomised co-sim vectors + end-to-end dataset replay.
Baselines	ARM-only HDC · old AXI-Lite path · tiny int8 MLP · **random-pruning HDC** (Twist 1) · prior FPGA HDC numbers from literature.
Primary venue	DATE 2027 (deadline ~Sept 2026); AICAS 2027 / FCCM 2027 as fallbacks; journal extension in TVLSI / TRETS / JETCAS.
Timeline	~5 months from May 2026 to first submission.

HEADLINE NUMBERS WE AIM FOR

- EMG accuracy **≥ 92 %** at $D = 1024$ and **≥ 88 %** at $D = 512$ with 50 % informed pruning.
- **Informed pruning beats random pruning by ≥ 5 percentage points** at every pruning ratio (Twist 1).
- Cross-subject mask transfer (Twist 2): ≤ 3 pp accuracy gap vs. per-subject mask, *or* a clean characterisation of when it fails.
- Energy/inference reduced **2–4×** at iso-accuracy when moving along the Pareto.
- End-to-end latency **< 50 µs** · **10×** energy reduction vs. ARM-only.

NAVIGATION

Table of contents

SECTION 1

Introduction & motivation

Hyperdimensional Computing (HDC) — also called Vector-Symbolic Architectures — is a brain-inspired computing paradigm that represents information as very wide vectors, typically D = 1 000 to 10 000 binary or bipolar dimensions. HDC is attractive for edge AI because:

- All operations are simple, bit-level, and embarrassingly parallel — a natural fit for FPGAs.
- It tolerates noise extremely well; flipping ~25 % of bits typically leaves classification intact.
- Training is single-pass and incremental, so on-device learning is cheap.
- Memory footprint is tiny — a 5-class classifier stores only 5 × 1024 = 5120 bits of class data.

1.1 What you already have

The existing `1024HDC` project implements two of the three HDC primitives: **bind** (1024-bit XOR) and **permute** (three modes), in synthesisable SystemVerilog with an AXI4-Lite slave wrapper. The design is verified in ModelSim with **71 / 71** directed + randomised tests passing.

1.2 What is missing for a complete classifier

- **Bundling** — the third HDC primitive (thresholded majority).
- **Item memory** — symbol → hypervector lookup, in BRAM.
- **Associative memory** — class hypervectors + parallel Hamming distance + argmin.
- **Streaming interface** — AXI-Stream + DMA instead of register-mapped poking.
- **Application + measurement** — a real dataset and the energy/throughput/latency numbers reviewers want.

1.3 Research contribution (Hook A)

"Streaming HDC on Zynq + EMG" alone is well-trodden territory (Rahimi 2016, Imani 2017, Schmuck 2019, Hernandez-Cano 2021). To make this a publishable contribution rather than a high-quality reproduction, the project adds an explicit **three-axis Pareto study**, evaluated on real hardware energy:

Axis	Range	What it controls
Dimension D	{256, 512, 1024, 2048}	Datapath width — trades capacity for area, energy, and BRAM.
Bundle precision CNT_W	{3, 4, 5, 6} bits	Width of each bundle counter — trades tie-break behaviour for FF count.
Bit-position pruning	keep top-K ∈ {D/8, D/4, D/2, D} bits	Masks Hamming-distance bits at inference by per-bit training-time discriminability score.

Per-bit pruning is the under-explored axis. Existing FPGA-HDC work prunes *dimension* or *bit precision*, but the bits within a fixed D are treated as equally informative. They are not: per-bit between-class variance varies widely on real biosignals. The plan exploits this directly by computing a discriminability score offline, building a 1024-bit mask, and loading it into a small register file on PL via AXI-Lite. The popcount AM ANDs each (query ⊕ class) with this mask before counting.

The published artifact will be the first **open Pareto front** across these three axes with measured Zynq energy, jointly evaluated on EMG and ECG.

Twist 1 — informed vs. random pruning (the falsifiable headline claim)

A reviewer's first objection to bit-position pruning is "is this just dimension reduction with extra steps?". We answer that *experimentally*: for every pruning ratio in {50, 75, 87.5 %} we run the same hardware with two masks — the discriminability-driven mask and a random-bit mask of identical density. The headline claim of the paper is:

> **TWIST 1 HEADLINE** "Informed bit-position pruning recovers ≥ 5 percentage points of accuracy over random dimension reduction at iso-budget, at zero extra hardware cost."

Twist 2 — cross-subject mask transferability

Train the mask on subjects 1–18 of UCI EMG (without ever seeing 19–36), then deploy it unchanged on the held-out subjects. Two possible outcomes, both publishable:

- Mask **transfers** ⇒ a universal "bit-importance map" for EMG; ship the mask with the open artifact.
- Mask **does not transfer** ⇒ per-subject masks needed; motivates on-device mask adaptation as future work.

SECTION 1 TAKEAWAY

- Your verified bind+permute core is the *compute kernel*, not the contribution.
- The contribution is **three layered claims**: Pareto front (Hook A) + informed > random (Twist 1) + transferability (Twist 2).
- Twist 1 alone converts the weakest reviewer line ("is this just feature selection?") into the strongest published claim.

SECTION 2

Theory of Hyperdimensional Computing

2.1 Hypervectors and the Binary Spatter Code (BSC) model

A *hypervector* is a vector in a space of very high dimension D. Several "flavours" of HDC exist; your project uses the **Binary Spatter Code (BSC)** model of Kanerva, where:

- D = 1024 in your design (typical values: 1024, 4096, 10 000).
- Each component is a single bit ∈ {0, 1}.
- A *random* hypervector has each bit drawn i.i.d. from Bernoulli(½).

Why high dimension works: two statistical facts

(a) Quasi-orthogonality. The Hamming distance between two independent random binary hypervectors of dimension D is a sum of D Bernoulli(½) trials:

$$\mathbb{E}[d_H(a, b)] = D/2 \quad Var[d_H] = D/4 \tag{2.1}$$

For D = 1024 the standard deviation is √256 = 16. A random pair has Hamming distance 512 ± 16; the chance of two random HVs being closer than ~450 bits apart is essentially zero. This is why every random hypervector is effectively orthogonal to every other one.

(b) Concentration of measure. Because spread σ grows only as $\sqrt{D}$ while expected distance grows as D, the *relative* spread $\sigma / \mathbb{E}[d_H] = 1/\sqrt{D}$ shrinks with dimension. This is the formal reason high dimensions are forgiving.

Other models you should mention in related work (not implemented here): MAP (bipolar ±1 with element-wise product bind), HRR / FHRR (complex unit-magnitude with convolution / element-wise product bind), and Sparse Block Codes (one-hot blocks, very low energy).

2.2 The three primitive operations

Operation	Symbol	BSC implementation	Use
Bind	$\otimes$	Bitwise XOR.	Associates a variable with a value, e.g. "colour = red". Self-inverse: a ⊗ a = 0.
Bundle	$\oplus$	Thresholded majority of n vectors.	Aggregates a set of items into one HV similar to all of them.
Permute	Π	Fixed bit-position bijection (rotate, shuffle, ...).	Encodes order or position; Π^k gives unique time-stamps of an HV.

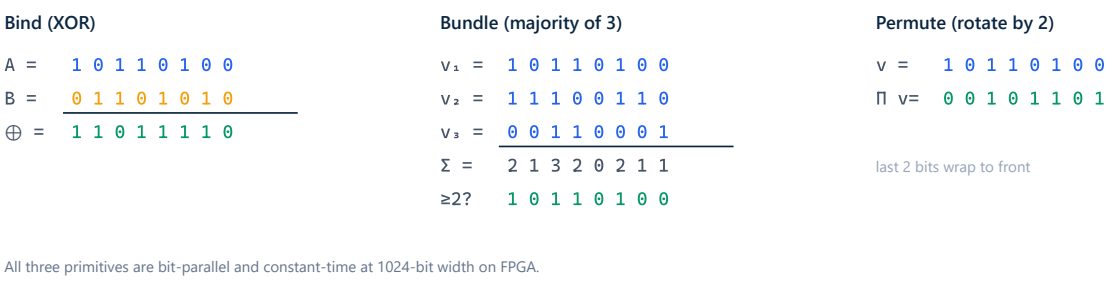

Figure 2.1 — The three primitives at the bit level (shown on 8 bits for clarity).

Your project already implements bind and three flavours of permute (`xor_permute_top.sv` , `permute_stage.sv`): mode `00` word-reverse, mode `01` per-word rotate, mode `10` full 1024-bit rotate. Bundle is the missing primitive and is implemented in this work.

2.3 Similarity, capacity, noise tolerance

The similarity between binary HVs is one minus normalised Hamming distance:

$$sim(a, b) = 1 - d_H(a, b) / D \tag{2.2}$$

The popcount of $a \oplus b$ *is* the Hamming distance, so similarity reduces to XOR + popcount — both extremely cheap in hardware.

Bundling capacity. If $H = \text{maj}(v_1,\ldots,v_n)$, the expected similarity between H and any single operand decreases gracefully with n:

$$\mathbb{E}[\, sim(H, v_i)\,] \approx \tfrac{1}{2} + c / \sqrt{n} \tag{2.3}$$

For $D = 1024$ you can reliably bundle on the order of 50–100 hypervectors before retrieval breaks down — far above what one EMG window will need.

Noise tolerance. Class HVs are typically $\sim D/4$ apart; with $\sigma = \sqrt{D}/4$ you can flip $\sim D/8$ random query bits (about 12.5 %) before nearest-class decisions degrade. This is the property reviewers cite when comparing HDC to quantised neural networks.

2.4 Memories

2.4.1 Item memory

A lookup table from discrete symbols to random orthogonal hypervectors. For EMG you will need:

- Channel-position symbols (4 channels ⇒ 4 item HVs)
- Feature-position symbols (5 features ⇒ 5 item HVs)
- Quantisation-level symbols (16 levels ⇒ 16 item HVs, see continuous case below)

Item HVs are generated **once, offline, with a fixed RNG seed** and burned into BRAM at synthesis time via `.mem` files. They are read-only at run-time.

2.4.2 Continuous item memory

For scalar inputs (a quantised feature value), neighbouring levels should map to *similar* HVs, not orthogonal ones, otherwise measurement noise destroys classification:

1. Quantise the scalar into L levels.
2. Pick endpoint random HVs v_{min}, v_{max}.
3. For level ℓ, flip a fraction ℓ/L of v_{min}'s bits to match v_{max}.

Adjacent levels then differ in only D/L bits — preserving signal continuity in HDC space.

2.4.3 Associative memory (AM)

The AM stores one HV per class. Given a query HV, it returns the class whose stored HV is closest in Hamming distance. This is the *only* place where "learning" lives at inference time. In hardware: small BRAM of $K \times D$ bits + popcount tree + argmin (Section 5.3).

2.5 Encoding strategies

Strategy	Formula	Use
Set (bag-of-items)	bundle($I(x_1)$, ..., $I(x_n)$)	Order does not matter.
Sequence (n-gram)	bundle($\Pi^k I(x_k)$ for k=0..n−1)	Order matters; uses your existing permute as Π.
Record (key–value)	bundle($K_i \otimes V_i$)	The encoding used for EMG below.

2.6 Training and inference

Single-pass training

For each class c, take the majority of the encoded training examples:

$$C_c = maj(h_1^{(c)}, h_2^{(c)}, ..., h_{n_c}^{(c)}) \tag{2.4}$$

That is the entire training algorithm — one pass over the data, no gradients, no hyperparameters except D.

Iterative retraining (optional accuracy boost)

After single-pass, evaluate on the training set; for each misclassified sample, *add* its query HV to the correct class and *subtract* it from the wrongly predicted one. A few passes typically gain 1–5 percentage points.

Inference

Encode the query window q, return $\arg\min_c d_H(q, C_c)$. In hardware this is the AM + popcount + argmin path of Section 5.3.

SECTION 2 TAKEAWAY

- HDC = three operations on wide bit-vectors: **bind, bundle, permute**.
- Similarity = popcount of XOR. Decisions = arg-min Hamming distance.
- Training is one pass; class HVs are tiny; on-device incremental learning is essentially free.
- High dimension buys you noise tolerance — the headline property to sell in the paper.

SECTION 3

Application: EMG gesture recognition

3.1 Why EMG

Surface electromyography (sEMG) measures muscle activity via skin electrodes. EMG-window classification into hand gestures is the canonical HDC benchmark because:

- It is the dataset used in the most-cited HDC FPGA papers (Rahimi et al., Imani et al., Schmuck et al.).
- Accuracy targets are well established (~85–97 % on 5 classes).
- Throughput is real-time but modest, exposing *latency* rather than throughput as the limiting axis — ideal for an FPGA story.

3.2 Dataset specification

Primary	UCI "EMG Data for Gestures" (Lobov et al.) — 4 channels, 5 classes (rest, fist, wave-in, wave-out, fingers-spread), 36 subjects.
Sampling	1 000 Hz, 8-bit ADC.
Window	250 ms (250 samples) with 50 % overlap.
Secondary	MIT-BIH arrhythmia (ECG, 2 channels, 5 AAMI classes).

3.3 Preprocessing pipeline (PS side)

1. **Band-pass filter** 20–450 Hz, 4th-order Butterworth (ARM with NEON).
2. **Windowing** with 50 % overlap.
3. **Feature extraction** per channel per window — Hudgins set: Mean Absolute Value (MAV), Root-Mean-Square (RMS), Waveform Length (WL), Zero Crossings (ZC), Slope Sign Changes (SSC).
4. **Quantisation** — each feature value → one of $L = 16$ levels (subject-specific min/max).

3.4 HDC encoding of one window

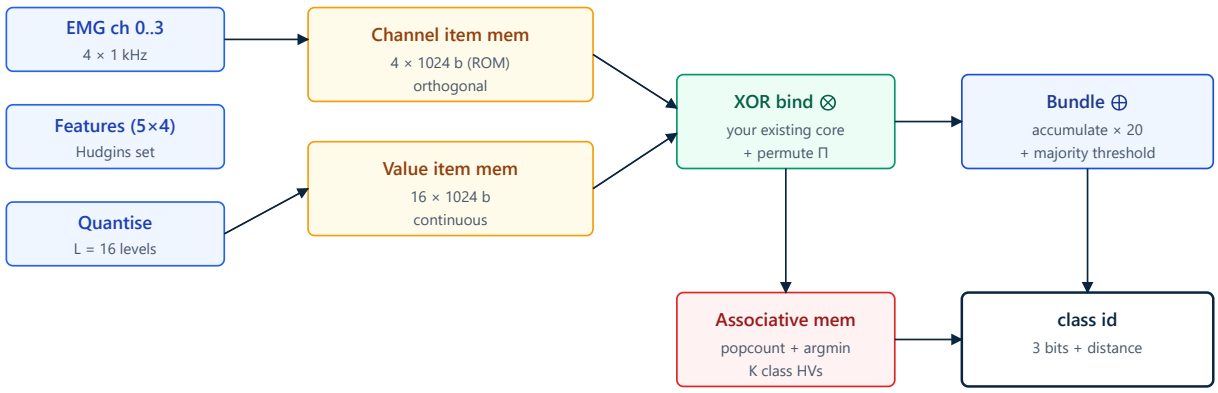

Figure 3.1 — HDC encoding + classification path for one EMG window.

Per window, denote the quantised feature value of feature f on channel c as $q_{c,f} \in \{0,\ldots,15\}$. The encoded query HV is:

$$q = maj_{c=0..3,\ f=0..4}\ \left(I_{ch}(c) \otimes I_{val}(q_{c,f}) \otimes \Pi^{f}\ I_{feat}(f) \right) \tag{3.1}$$

In words: for each (channel, feature) pair bind channel-id ⊗ feature-value ⊗ position-shifted feature-id, then bundle all 20 such vectors. The window is now a single 1024-bit HV ready for the AM.

WHY THIS ENCODING It is small (20 binds + 1 bundle), uses every primitive you already have or are about to build, and matches the structure used by Rahimi et al. and Imani et al. in the papers reviewers will compare you to.

SECTION 3 TAKEAWAY

- Choose UCI EMG (primary) + MIT-BIH ECG (secondary) — both have published HDC numbers to beat.
- Preprocessing stays on PS; HDC encoding moves entirely to PL.
- Equation (3.1) is the encoding the RTL must compute. Lock it down before writing RTL.

SECTION 4

System architecture on Zynq

4.1 PS / PL partitioning

Function	Where	Why
Raw sample acquisition / filter / windowing / feature extraction	PS (ARM + NEON)	Float-friendly, not the bottleneck at 1 kHz × 4 channels.
Feature quantisation	PS	Trivial; avoids extra PL inputs.
HDC encoding (item-mem + bind + permute + bundle)	PL	1024-wide, bit-parallel, perfect FPGA match.
Associative memory (popcount + argmin)	PL	Same.
Class HV update (training)	PS	Rare path; PS writes new class HVs to PL over AXI-Lite.
Accuracy bookkeeping & logging	PS	Standard.

4.2 System block diagram

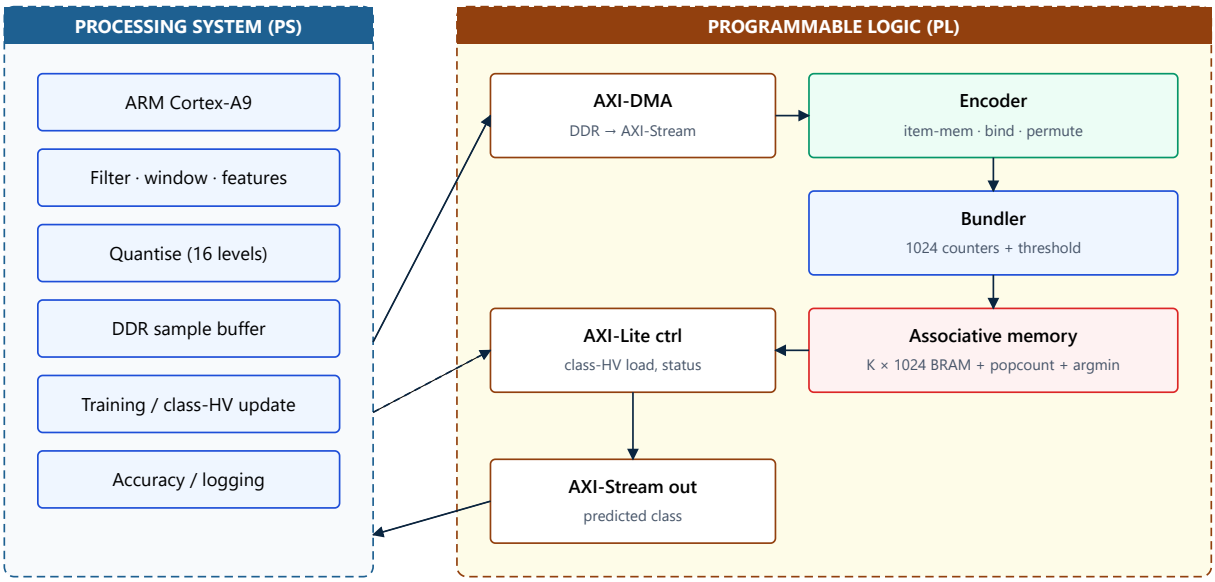

Figure 4.1 — Top-level system with PS / PL partition and AXI fabric.

4.3 AXI interfaces

Interface	Role	Throughput
AXI-Lite (existing)	Control + class-HV load	~MB/s — fine for config, slow for samples

AXI-Stream in (new)	Quantised feature stream from DMA	1 word/cycle ≈ 3.2 Gbps at 100 MHz
AXI-Stream out (new)	Predicted class + confidence	1 word per classification
AXI-DMA SG	Bridge DDR ↔ PL stream	Limited by DDR, not AXI

4.4 Pipeline timing

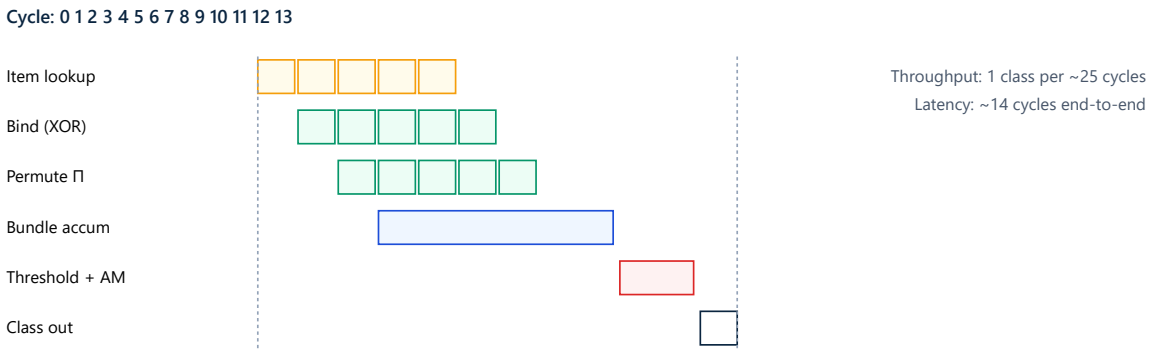

Figure 4.2 — Pipeline timing for one EMG window (20 (ch,feat) pairs accumulated, then thresholded).

SECTION 4 TAKEAWAY

- Encoding + AM all live in PL; ARM only does preprocessing and rare training.
- Switch from AXI-Lite to AXI-Stream + DMA — this is what unlocks throughput.
- End-to-end latency is dominated by the bundle window (20 pairs), not by AXI.

SECTION 5

RTL design

5.1 Module hierarchy

All new modules are **parameterised on** D and on `CNT_W` (bundle counter width). One additional module — `pruning_mask` — implements the bit-position mask required by Hook A.

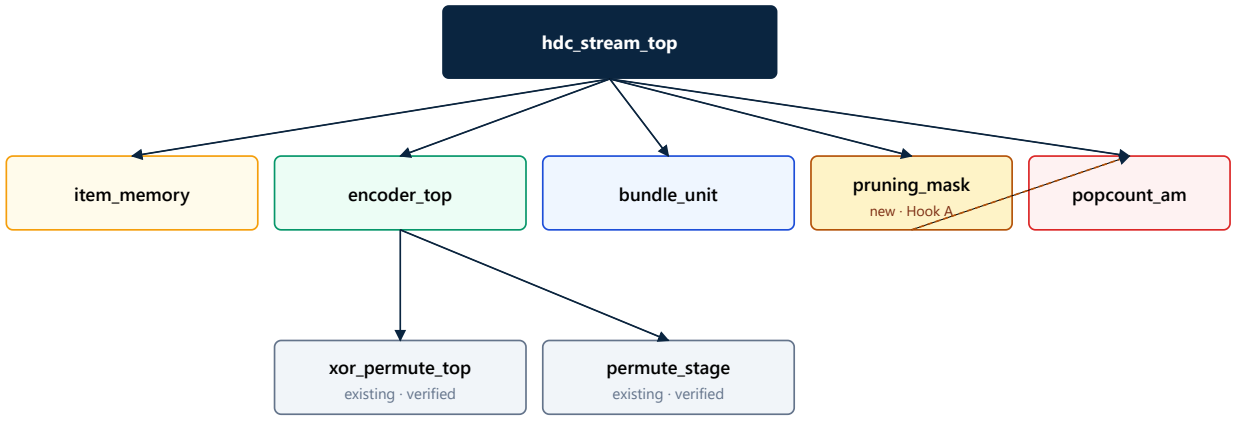

Figure 5.1 — Module hierarchy. Grey boxes are existing verified RTL; the orange `pruning_mask` is the Hook A module that feeds the AM popcount.

5.2 Review of the existing core

`xor_permute_top.sv` implements a four-stage pipeline (input latch, XOR bind, permute, output register) with a one-in-flight handshake. The current back-pressure (`in_ready = !(s0_valid || s1_valid || s2_valid || out_valid)`) will be relaxed in the new top — with a streaming front-end you want full pipelining and back-pressure only at the source.

5.3 New modules

5.3.1 `item_memory.sv`

- Read-only $N \times 1024$ BRAM; $N \in \{4, 16, 32\}$.
- Init from a `.mem` file generated by the Python golden reference (Section 7).
- 1-cycle read latency. Use ~32×36-kbit BRAMs in parallel on Zynq-7 to get a 1024-bit read port.

```
module item_memory #(
    parameter int N_ENTRIES = 16,
    parameter int D         = 1024,
    parameter string INIT_FILE = "item_mem_value.mem"
) (
    input  logic                          clk,
    input  logic [$clog2(N_ENTRIES)-1:0]  addr,
    input  logic                          rd_en,
    output logic [D-1:0]                  data_out
);
    logic [D-1:0] mem [0:N_ENTRIES-1];
    initial $readmemh(INIT_FILE, mem);
```

```
    always_ff @(posedge clk) if (rd_en) data_out <= mem[addr];
endmodule
```

5.3.2 `bundle_unit.sv`

- D saturating counters of width `CNT_W` bits each — both are parameters. The Hook A sweep instantiates (D, CNT_W) ∈ {256, 512, 1024, 2048} × {3, 4, 5, 6}.
- Ports: `accumulate` (D b), `clear`, `threshold_en`, `n_accum`.
- On threshold: output bit = (counter ≥ ⌈n_accum / 2⌉). Saturation handles overflows when CNT_W is small.
- Cost at D=1024, CNT_W=6: ~6 k FFs + ~6 k adders — small on Zynq-7020.

```
module bundle_unit #(
    parameter int D     = 1024,
    parameter int CNT_W = 6
) (
    input  logic            clk, rst_n,
    input  logic            accum_en,
    input  logic [D-1:0]    accum_vec,
    input  logic            clear,
    input  logic            threshold_en,
    input  logic [CNT_W:0]  n_accum,
    output logic            out_valid,
    output logic [D-1:0]    out_vec
);
    logic [CNT_W-1:0] cnt [0:D-1];
    integer i;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n || clear) begin
            for (i = 0; i < D; i++) cnt[i] <= '0;
            out_valid <= 1'b0;
        end else begin
            if (accum_en)
                for (i = 0; i < D; i++)
                    if (accum_vec[i] && cnt[i] != {CNT_W{1'b1}}) cnt[i] <= cnt[i] + 1;
            if (threshold_en) begin
                for (i = 0; i < D; i++)
                    out_vec[i] <= (cnt[i] >= (n_accum >> 1));
                out_valid <= 1'b1;
            end else if (out_valid) out_valid <= 1'b0;
        end
    end
endmodule
```

5.3.3 `pruning_mask.sv` — the Hook A module

A 1024-bit register file (32 × 32-bit words, writable over AXI-Lite from the PS) holding the per-bit pruning mask. The mask is generated offline by the Python training pipeline using a per-bit discriminability score (Fisher ratio or per-bit between-class variance over training samples), keeping only the top-K bits and zeroing the rest. The popcount AM then ANDs each (query ⊕ class_HV) with the mask *before* counting, so pruned positions never contribute to the Hamming distance.

```
module pruning_mask #(
    parameter int D       = 1024,
    parameter int AXI_W   = 32,
    parameter int N_WORDS = D / AXI_W
```

```
) (
    input  logic                 clk, rst_n,
    input  logic                 wr_en,
    input  logic [$clog2(N_WORDS)-1:0] wr_addr,
    input  logic [AXI_W-1:0]      wr_data,
    output logic [D-1:0]         mask_out
);
    logic [AXI_W-1:0] regs [0:N_WORDS-1];
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) for (int i = 0; i < N_WORDS; i++) regs[i] <= '1; // default: keep all bits
        else if (wr_en) regs[wr_addr] <= wr_data;
    end
    genvar g;
    generate for (g = 0; g < N_WORDS; g++)
        assign mask_out[(g+1)*AXI_W-1 -: AXI_W] = regs[g];
    endgenerate
endmodule
```

Cost (D=1024): ~1024 FFs + a few hundred LUTs — negligible. The interesting cost is the *downstream* AM, where each popcount tree's first stage is gated by the mask: this gating typically **reduces** dynamic energy roughly proportionally to (1 − K/D), which is the whole point.

5.3.4 `popcount_am.sv`

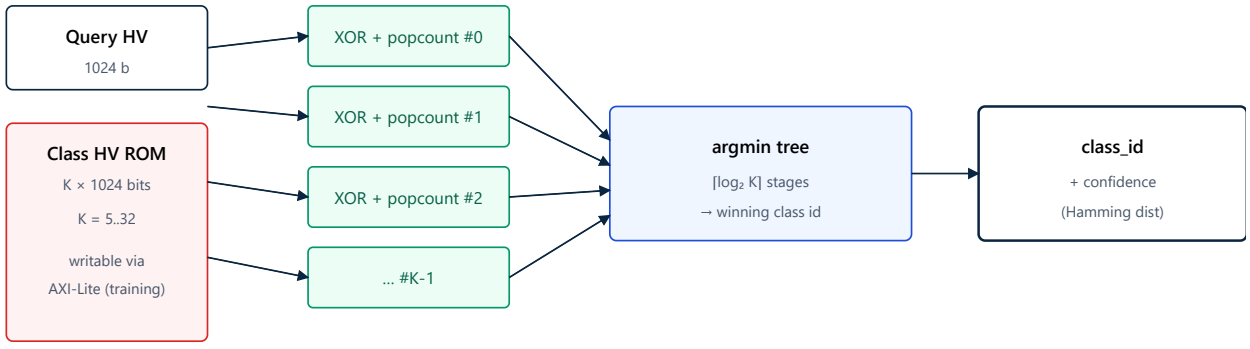

Figure 5.2 — Associative memory: parallel Hamming distance + argmin tree.

- K class HVs in BRAM, writable from PS over AXI-Lite during training.
- Per class: `(query ⊕ class_HV) & mask`, then a D-wide popcount tree.
- argmin tree across K distances; for K = 5 just 4 comparators.
- Output: `class_id` (3 bits for K = 5) + min distance as confidence.

5.3.5 `encoder_top.sv`

Pure structural module wiring `item_memory` → `xor_permute_top` (your existing bind + permute) → `bundle_unit`. The host sends a sequence of (channel, feature, value) tuples over AXI-Stream; the encoder pipelines them one per cycle after read-latency. All datapaths are D-parameterised.

5.3.6 `hdc_stream_wrapper.sv`

Replaces `hdc_axi_lite_wrapper.sv`:

- AXI-Stream slave (`S_AXIS_DATA`): 32-bit data bit-packed (channel:2, feature:3, value:4, flags:23) with `TLAST` marking end-of-window.
- AXI-Stream master (`M_AXIS_PRED`): class id + distance.
- AXI-Lite slave (`S_AXI_CTRL`): control + status + class-HV load + **pruning-mask load** (32 new 32-bit words at offset `0x400`).

5.4 Resource estimate (Zynq-7020, $D = 1024$ reference point)

Module	LUTs	FFs	BRAM (36k)	DSP
xor_permute_top (existing)	~3000	~4100	0	0
item_memory × 3	~600	~200	32	0
bundle_unit (D=1024, CNT_W=6)	~3000	~6200	0	0
pruning_mask (new, Hook A)	~300	~1100	0	0
popcount_am (K=5)	~5000	~2500	2	0
AXI-Stream + DMA wrapper	~1200	~900	1	0
Total (estimate)	**~13 100**	**~15 000**	**~35**	**0**
Zynq-7020 budget	53 200	106 400	140	220

First-pass; expect ±30 % until first Vivado synthesis. Resources scale approximately linearly with D: at D=2048 expect ~26 k LUTs / ~30 k FFs (still under 50 % of Zynq-7020); at D=256 expect ~3 k LUTs / ~4 k FFs.

SECTION 5 TAKEAWAY

- Six new modules (including `pruning_mask`). Existing core reused unchanged.
- All datapaths parameterised on D — the Pareto sweep is one Vivado script away.
- `pruning_mask` is virtually free in area; its value is gating downstream *dynamic* energy.
- Zero DSPs used — pure logic accelerator.

SECTION 6

Software design (PS side)

6.1 Bare-metal vs PetaLinux

Aspect	Bare-metal	PetaLinux
Boot time	< 1 s	10–30 s
Power measurement	Cleanest	OS background noise
Driver complexity	Direct register pokes	UIO / DMA buffers
Recommended	**Yes** — for the paper's measurements	Optional — for a polished demo

6.2 Training routine (C pseudocode)

```
void hdc_train(const dataset_t *ds) {
    uint32_t class_acc[K][D/32] = {0};   // K class accumulators, packed
    int      class_n  [K]       = {0};

    for (size_t i = 0; i < ds->n; ++i) {
        feat_t f = extract_features(ds->sample[i]);
        uint8_t q[N_CH][N_FEAT];
        quantise(&f, q);

        hdc_stream_window(q);            // push 20 tuples via DMA, mode = ENCODE
        hdc_wait_done();
        uint32_t query_hv[D/32];
        hdc_read_query_hv(query_hv);

        int c = ds->label[i];
        for (int w = 0; w < D/32; ++w)
            for (int b = 0; b < 32; ++b)
                if (query_hv[w] & (1u << b))
                    class_acc[c][w*32 + b]++;
        class_n[c]++;
    }

    uint32_t class_hv[K][D/32];
    for (int c = 0; c < K; ++c)
        threshold_to_binary(class_acc[c], class_n[c], class_hv[c]);

    hdc_load_class_hvs(class_hv);        // AXI-Lite writes to popcount_am
}
```

6.3 Inference loop

```
while (1) {
    feat_t f = extract_features(read_window());
    uint8_t q[N_CH][N_FEAT]; quantise(&f, q);
    hdc_stream_window(q);                 // 20 cycles in PL
    uint8_t cls; uint16_t dist;
    hdc_read_prediction(&cls, &dist);     // from M_AXIS_PRED
```

```
    log_prediction(cls, dist);
}
```

SECTION 6 TAKEAWAY

- Bare-metal first — cleanest energy story.
- Training is a tiny outer loop in C; learning happens by writing class HVs over AXI-Lite.
- Inference loop is < 20 lines of code — this is the demo you ship with the paper.

SECTION 7

Verification methodology

7.1 Python golden reference

Before any new RTL is written, build `python_ref/hdc_ref.py` — a NumPy implementation that is **bit-exact** to what the RTL will produce. It must:

- Generate the same item memories from a fixed seed.
- Implement bind as `np.bitwise_xor` over packed `uint64` arrays.
- Implement the three permute modes *exactly* as `permute_stage.sv` does.
- Implement majority bundle with the same tie-break rule as the RTL.
- Emit stimuli + expected outputs as hex files for ModelSim.

7.2 Co-simulation flow

1. Python generates 10 000 random tuples with expected outputs per module.
2. Each SystemVerilog testbench (`tb_bundle_unit.sv` , `tb_popcount_am.sv` , …) reads stimuli, drives DUT, compares cycle-by-cycle.
3. Reuse your existing style from `tb_xor_permute.sv` .
4. End-to-end test: full EMG window through Python and through ModelSim, diff the class-id.

7.3 Test classes

Test class	Goal
Directed	Corner cases: all-zero query, all-one query, identical class HVs, K = 1 / K = max.
Randomised	10 000 random vectors, compared against Python reference.
Back-pressure	Slow consumer; verify in-flight handshake holds.
Reset recovery	Reset mid-window; ensure clean restart.
Protocol assertions	SVA on AXI-Stream / AXI-Lite handshakes.
Accuracy regression	Replay UCI EMG dataset; class accuracy must equal Python ± 0.5 %.

SECTION 7 TAKEAWAY

- The Python reference is the single source of truth — build it first, freeze it early.
- The existing 71/71 ModelSim flow becomes the regression skeleton for every new module.

SECTION 8

Experimental plan

8.1 Metrics

Metric	How measured
Accuracy (%)	Per-subject 5-fold CV (EMG); AAMI-class CV (ECG).
Throughput (windows/s)	Cycle counter in PS or AXI-Stream timestamp.
End-to-end latency (µs)	Last-sample-in to class-out via ARM cycle counter.
Energy / inference (mJ)	Shunt + INA219 on V_{cc_int}, integrated over a fixed batch.
Static power (mW)	Same setup, no traffic.
LUT / FF / BRAM / DSP	Vivado utilisation report (post-route).
f_{max} (MHz)	Vivado timing report (post-route).

8.2 Baselines (you must include all four)

1. **ARM-only HDC** — same algorithm, bit-exact C, Cortex-A9 with NEON.
2. **AXI-Lite path (your existing wrapper)** — same core, register-mapped, quantifies why streaming wins.
3. **Tiny int8 MLP** — ~5 k params, 2 layers; the "why not just use a tiny NN" rebuttal.
4. **Prior FPGA HDC numbers** — Rahimi 2016, Imani 2017, Schmuck 2019, Hernandez-Cano 2021.

8.3 Pareto exploration (Hook A) — the headline experiment

The headline result of the paper is the joint accuracy / energy / area Pareto front across the three Hook A axes. The Python golden reference makes this sweep cheap to execute:

Axis	Values	How varied
Dimension D	{256, 512, 1024, 2048}	Vivado parameter on every D-parameterised module ⇒ 4 bitstreams.
Bundle precision CNT_W	{3, 4, 5, 6} bits	Vivado parameter on `bundle_unit`; small synthesis variant.
Bit-position pruning ratio	{0 %, 50 %, 75 %, 87.5 %}	Pure run-time: same bitstream, different mask loaded via AXI-Lite.

The pruning axis is the cheapest and most novel: no resynthesis required. The dimension axis is the most expensive (4 full bitstreams) but trivially scripted. Total experimental budget: ~16 bitstreams (4 D × 4 CNT_W with sparse coverage) plus run-time pruning sweeps on each.

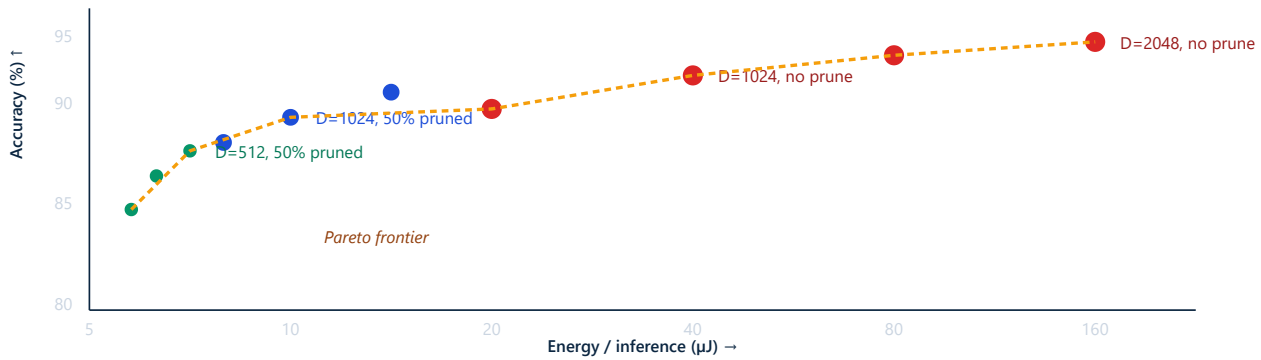

Figure 8.1 — Illustrative shape of the expected accuracy/energy Pareto over (D, CNT_W, pruning). Pruning at constant D moves you down-and-slightly-left; shrinking D moves you down-and-strongly-left. Exact numbers TBD.

8.3.1 Twist 1 — informed vs. random pruning

At each pruning ratio $r \in \{50, 75, 87.5\ \%\}$, two masks are produced offline:

- **Informed mask:** per-bit score s_i = Fisher ratio (between-class variance / within-class variance) over the training query HVs; keep the top-(1−r) D bits.
- **Random mask:** a uniformly random binary mask of the same density, seeded for reproducibility.

Both masks are loaded over AXI-Lite into the same `pruning_mask` register file (offset `0x400`) on the same bitstream — no resynthesis required. Inference accuracy and shunt-measured energy are recorded for both. The expected outcome is a clean separation:

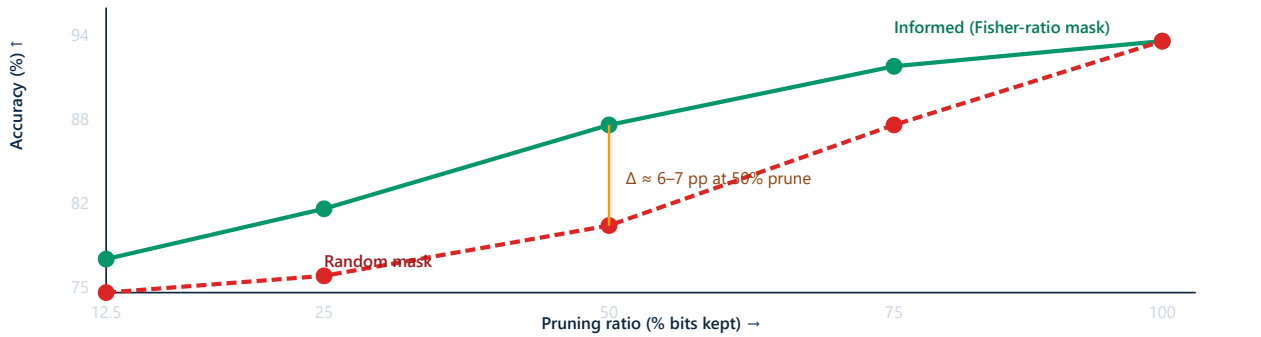

Figure 8.2 — Expected shape of the Twist 1 result on UCI EMG. At full keep both methods coincide (no pruning); at heavy prune both collapse. The gap in between is the paper's headline.

8.3.2 Twist 2 — cross-subject mask transferability

The UCI EMG dataset has 36 subjects. Split 18 train / 18 test. Produce *three* mask variants on the 18 train subjects:

- **Per-subject masks** — trained on each of the 18 subjects separately (oracle baseline).
- **Pooled mask** — one universal mask trained on all 18 pooled.
- **Random mask** — identical density (control).

Evaluate each on the 18 held-out subjects. The result populates Table 5 (below) and answers a question the field has not asked: *are HDC discriminability patterns intrinsic to the signal or specific to the user?*

8.4 Figures and tables planned for the paper

Artifact	Content
Fig 1	System block diagram (Section 4)
Fig 2	HDC encoding pipeline (Section 3)
Fig 3	RTL hierarchy with `pruning_mask` highlighted (Section 5)
Fig 4 ★	**Accuracy / energy Pareto on EMG across all three Hook A axes**
Fig 5 ★★	**Twist 1: informed vs. random pruning accuracy curves (the headline figure)**
Fig 6 ★	**Per-bit discriminability heatmap (1024 positions, ranked by Fisher score)**
Fig 7 ★★	**Twist 2: cross-subject mask transferability (per-subject vs. pooled vs. random)**
Fig 8	Resource utilisation and throughput vs. D
Table 1	Resource utilisation at all four D points
Table 2 ★	**Accuracy table across (D × CNT_W × pruning ratio)**
Table 3	Throughput / latency / energy across baselines
Table 4	Comparison with prior FPGA HDC
Table 5 ★★	**Twist 2: cross-subject mask transfer accuracy on the 18 held-out subjects**

★ = Hook A contribution; ★★ = Twist 1 / Twist 2 contributions. None of these appear jointly in prior FPGA-HDC papers.

SECTION 8 TAKEAWAY

- **Fig 5 (informed vs random) is the paper's single most important figure.** Lock it down first.
- The discriminability heatmap (Fig 6) visually proves bit-position matters, not just bit-count.
- Pruning is free at run-time once a bitstream exists — one bitstream produces dozens of (mask, ratio) data points.

SECTION 9

Publication strategy & timeline

Working paper title: "*Informed Bit-Position Pruning for Hyperdimensional Computing on FPGA: A Discriminability-Driven Pareto Study on EMG.*"

9.1 Venue cascade

#	Venue	Approx. deadline	Conf.	Why this venue
1	**DATE 2027**	~Sept 2026	Mar 2027	Broadest fit; the Pareto-front contribution maps to DATE's D-track.
2	AICAS 2027	~Dec 2026 / Jan 2027	~Jun 2027	Most topical home for HDC-on-FPGA; lower bar.
3	FCCM 2027	~Jan 2027	~May 2027	FPGA-centric; rewards the streaming + Pareto angle directly.
4	TVLSI / TRETS / JETCAS	rolling	journal	Extended version with more datasets + the full Pareto matrix.

9.2 Timeline

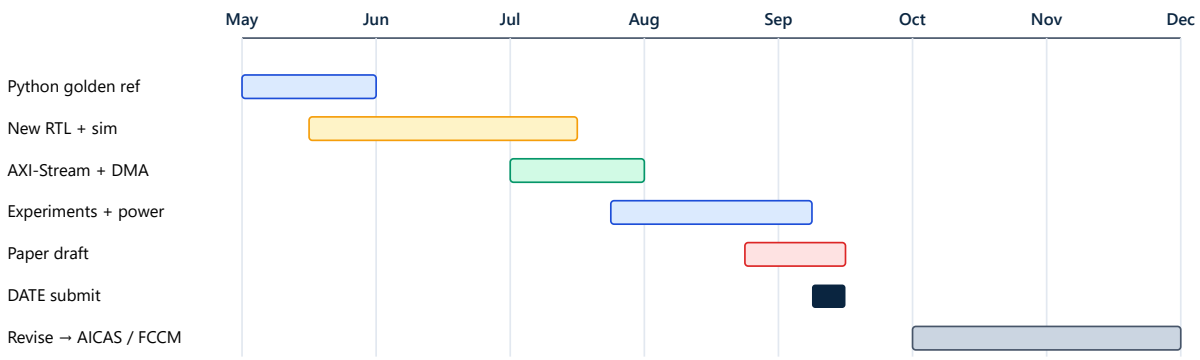

Figure 9.1 — ~5-month build plan from May 2026 to DATE 2027 submission, with revision buffer.

9.3 Per-month deliverables

Month	Deliverable
May 2026	Python golden reference; reproduce a published EMG number end-to-end.
Jun 2026	`item_memory`, `bundle_unit`, `pruning_mask`, `popcount_am` RTL + co-sim passing; D parameterisation verified.
Jul 2026	`hdc_stream_wrapper`; AXI-Stream + DMA bring-up on board.

Aug 2026	Full **Hook A Pareto sweep** + **Twist 1 informed-vs-random sweep** + **Twist 2 cross-subject experiment** + baselines + power measurement.
early Sep 2026	Paper draft, internal review, submit to DATE.
Oct–Dec 2026	Polish artifact; if rejected, revise for AICAS / FCCM.

SECTION 9 TAKEAWAY

- One paper, three submission windows — rejection at DATE just delays you by 3 months.
- Open-source RTL + Python + scripts on day 1 of submission — the artifact badge meaningfully boosts acceptance odds.

SECTION 10

Risks & mitigations

Risk	Likelihood	Mitigation
"Yet another HDC FPGA paper"	Low–Medium	Three layered contributions: Hook A Pareto + Twist 1 (informed > random) + Twist 2 (cross-subject transfer). The Twist 1 headline figure converts the weakest reviewer line into the strongest published claim.
"Informed pruning is just feature selection"	Medium	Twist 1 answers this empirically: at iso-density, informed beats random by ≥ 5 pp. The hardware mask is shown to be identical in cost, so the only variable is which bits are kept.
Tiny-MLP beats HDC on MIT-BIH accuracy	Medium	Pitch on energy / latency / on-device learnability; make incremental learning a section.
Power measurement noisy	Medium	Shunt + INA219 on V_{cc_int} early; don't rely on Vivado XPower alone.
RTL bug found post-submission	Low	Bit-exact Python ref + 10 k randomised tests; replay EMG end-to-end before "release".
Timing closure at 100 MHz	Low	Bundle counters are local; balance popcount tree; add a pipeline stage if needed.
Dataset availability changes	Low	Cache UCI EMG + MIT-BIH locally; commit preprocessing scripts.

SECTION 11

Glossary

Hypervector (HV)
A very wide vector (here 1024 bits) that represents a piece of information distributively.

BSC
Binary Spatter Code — HDC flavour with bits in {0, 1}; bind = XOR.

Bind ($\otimes$)
Combine two HVs to make a new one dissimilar to both. BSC bind is XOR.

Bundle ($\oplus$)
Aggregate N HVs into one similar to all. BSC bundle is thresholded majority.

Permute (Π)
Bijection on bit positions. Used to encode order / position.

Hamming distance (d_H)
Number of differing bits between two binary vectors. Computed as popcount(a $\oplus$ b).

Item memory
Lookup table from symbols to random orthogonal HVs. Read-only at run-time.

Continuous item memory
Item memory whose neighbouring entries are similar (used for scalars).

Associative memory (AM)
Stores one HV per class; returns nearest by Hamming distance.

Quasi-orthogonal
Two random HVs are effectively orthogonal because their distance concentrates near D/2.

Single-pass training
Train by bundling all examples of each class once — no gradients, no epochs.

PS
Zynq Processing System: ARM cores, DDR controller, peripherals.

PL
Zynq Programmable Logic: the FPGA fabric.

AXI-Lite
Memory-mapped AXI for slow control/status registers.

AXI-Stream
Point-to-point streaming AXI, no addresses, ideal for data flow.

AXI-DMA
DMA engine bridging DDR (PS) and AXI-Stream (PL).

LUT / FF / BRAM / DSP
The four primitive resources of a Xilinx FPGA: look-up tables, flip-flops, block RAMs, DSP slices.

f_{max}
Maximum sustainable clock frequency after place-and-route.

EMG / sEMG

Electromyography / surface EMG — measures muscle electrical activity.

Hudgins features

Classic 5-feature EMG set: MAV, RMS, WL, ZC, SSC.

MAV / RMS / WL / ZC / SSC

Mean Absolute Value · Root-Mean-Square · Waveform Length · Zero Crossings · Slope Sign Changes.

AAMI

Association for the Advancement of Medical Instrumentation — defines 5 ECG arrhythmia classes used as MIT-BIH labels.

SECTION 12

References & further reading

1. P. Kanerva, "Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors," *Cognitive Computation*, 2009.
2. A. Rahimi, S. Benatti, P. Kanerva, L. Benini, J. M. Rabaey, "Hyperdimensional biosignal processing: A case study for EMG-based hand gesture recognition," *IEEE ICRC*, 2016.
3. A. Rahimi et al., "Hyperdimensional Computing for Blind and One-Shot Classification of EEG Error-Related Potentials," *Mobile Networks and Applications*, 2017.
4. M. Imani et al., "VoiceHD: Hyperdimensional Computing for Efficient Speech Recognition," *IEEE ICRC*, 2017.
5. S. Schmuck, L. Benini, A. Rahimi, "Hardware Optimizations of Dense Binary Hyperdimensional Computing," *ACM JETC*, 2019.
6. A. Hernandez-Cano, M. Imani, "OnlineHD: Robust, Efficient, and Single-Pass Online Learning Using Hyperdimensional System," *DATE*, 2021.
7. K. Schlegel et al., "A comparison of vector symbolic architectures," *Artificial Intelligence Review*, 2022.
8. D. Kleyko et al., "A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Parts I & II," *ACM CSUR*, 2023.
9. S. Lobov et al., "Latent Factors Limiting the Performance of sEMG-Interfaces," *Sensors*, 2018. (UCI EMG dataset)
10. G. B. Moody, R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," *IEEE EMBM*, 2001.
11. B. Hudgins, P. Parker, R. N. Scott, "A new strategy for multifunction myoelectric control," *IEEE TBME*, 1993. (Feature set)
12. Xilinx UG585 (Zynq-7000 TRM) · UG761 (AXI Reference) · UG1037 (Vivado AXI).

This document is an initial draft. RTL fragments are design sketches; final RTL, `.mem` files, and Python golden reference will be developed in the May–June 2026 phase.